Victoria

Generated by Doxygen 1.9.8

1 Victoria — ∗"Whatever, I do what I want!"∗

> ∗"You're not my real mom, Unity. You can't tell me where to put my files."∗ — Victoria, probably

1.1 </blockquote>

You've been there. You double-click a `.unitypackage`. Unity opens its little import dialog. You click **Import**. And suddenly your project has a `Assets/SomeRandom/Vendor/DeepNested/Scripts/Utilities/Helpers/Manager/` folder shoved right into your carefully curated directory structure. No warning. No negotiation. Just files, **wherever the package author felt like putting them** on the day they made it.

Well. Not anymore.

Victoria is the ∗"Whatever, I do what I want"∗ `.unitypackage` importer. She doesn't care what the package author intended. She doesn't care about their folder structure. She reads the package, shows you what's inside, and then — and this is the important part — **lets YOU decide where everything goes**.

1.2 Features

1.2.1 Browse Before You Commit — ∗"I'm not importing that."∗

Victoria opens a full tree view of every folder and file inside the `.unitypackage`. You can poke around, judge the structure, and decide you absolutely do not want `TheirScripts/Utils/StringExtensions.cs` cluttering up your project before you've even imported a single byte. Checkboxes let you select exactly what you want and skip the rest.

1.2.2 Drag & Drop to Wherever You Please — ∗"I'll put it HERE. Because I want to."∗

See that script? Drag it to your `Scripts/` folder. See that texture? Drop it in `Art/Environment/`. You are not a victim of someone else's folder conventions. The right-hand panel shows your actual project structure, and you can drop package assets directly into any folder you like. Total control. Zero compromises.
The drag and drop system is internally called `SleurEnPleur`, which is Dutch for "drag and dump". You're welcome.

1.2.3 Asset Preview — ∗"Let me look at it first."∗

Click any file in the package tree and Victoria shows you what's inside before you import it. She supports:

- **Text files** — `.cs`, `.json`, `.md`, `.txt`, `.uss`, `.uxml`, `.asmdef`, `.asset`, `.meta` — full scrollable source code view
- **Images** — renders the thumbnail preview baked into the package
- **Audio** — `.mp3`, `.wav`, `.ogg` — play button included, no questions asked
- **3D Models** — `.fbx`, `.dae`, `.obj`, `.3ds`, `.dxf` — she sees them, she acknowledges them

If there's no preview available she'll tell you straight up. No drama. Well, some drama.

1.2.4 Search — ∗"Where is it?! I KNOW it's in here."∗

The package tree has a search bar. Type a name, Victoria runs a breadth-first search through the entire package structure and highlights the matching nodes. Finding that one script buried six folders deep has never been more satisfying.

1.2.5 Runtime Import — *"I don't need the editor. I NEVER needed the editor."*

Yes, Addressables exist. Yes, AssetBundles exist. Victoria is aware. Victoria imports `.unitypackage` files at runtime anyway, on a Tuesday afternoon, just because she can.
This is a pure API. No file explorer, no window, no cross-platform file picker nonsense. You give Victoria a path. She does the rest.

```
var importer = new VictoriaRuntimeImporter();
await importer.ImportAsync("path/to/your.unitypackage");
// Files are now in Application.persistentDataPath.
// Load them however you want. Victoria is not your mom.
```

Once she's done, everything is sitting in `Application.persistentDataPath` and Victoria has moved on with her life. What happens next is entirely your business. Load a `.fbx` with your favorite runtime model loader. Play back audio. Hotpatch your game with fresh assets without touching a build. Use whatever third-party library you want for whatever asset type you want. She does not care. She put the files on disk. Her job is done.

1.3 Installation

Add the package via the Unity Package Manager using the git URL:

```
https://github.com/HamerSoft/Victoria.git
```

Or add it directly to your `manifest.json`:

```
{
  "dependencies": {
    "com.hamersoft.victoria": "https://github.com/HamerSoft/Victoria.git"
  }
}
```

Unity 2019.4 or higher. Victoria has standards.

1.4 Usage

1. Go to **Tools** → **HamerSoft** → **Victoria** → **Import Package**
2. Select your `.unitypackage` file
3. The window opens, titled *"Whatever, I do what I want!"*
4. Browse the package tree on the left. Search if you need to. Click files to preview them on the right.
5. Check the files and folders you actually want.
6. Drag them to wherever you want them in your project's folder structure on the bottom-right panel.
7. Hit **Import**.
8. Bask in the glory of a project structure that is *yours*.

1.5 Why "Victoria"?

Because sometimes a package author puts their files in `Assets/TheirCompanyName/TheirProduct↩Name/Version/1_0/Runtime/Scripts/Core/` and you just want to scream *"WHATEVER! I DO WHAT I WANT!"* and put them in `Scripts/` like a normal person.
Victoria gets it. Victoria is you.

1.6 Contributing

PRs welcome. Victoria has no gatekeepers. She does what she wants — and so can you.

- `Documentation`
- `Changelog`
- `Report Issues`

Made by `HamerSoft` — inspired by a very specific energy.

2 0.1.0 (2026-05-16)

2.0.1 Features

- add runtime import, but stylesheet is missing some sizing (1b1d74d)
- finish runtime import (75b7b8a)

3 LICENSE

4 Namespace Index

4.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

5 Hierarchical Index

5.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

6 Class Index

6.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

7 Namespace Documentation

7.1 HamerSoft Namespace Reference

7.2 HamerSoft.Victoria Namespace Reference

Classes

- class UnityPackage

  Represents a parsed `.unitypackage` file. Exposes the package's asset tree and provides methods for loading individual assets on demand and importing them to disk. Dispose when done to release all cached Unity objects and the audio source.

7.3 HamerSoft.Victoria.Core Namespace Reference

7.4 HamerSoft.Victoria.Core.Audio Namespace Reference

Classes

- interface IAudioSource

  AudioSource used for previewing audio data.

7.5 HamerSoft.Victoria.Core.Extractor Namespace Reference

Classes

- class Extractor

  Extractor class to load a .unitypackage file into memory.

7.6 HamerSoft.Victoria.Core.Extractor.Nodes Namespace Reference

Classes

- class Asset

  Class representing an asset.
- class FileSystemNode

  A Node implementation that wraps an existing file-system entry. Constructed from a DirectoryInfo (non-leaf) or a FileInfo (leaf). Unlike package-extracted nodes, file-system nodes already exist on disk and cannot be written out.
- class Folder

  Class representing a folder of the imported .unityPackage.
- class Node

  base class for representing nodes in the file-tree

7.7 HamerSoft.Victoria.Core.Import Namespace Reference

7.8 HamerSoft.Victoria.Core.Search Namespace Reference

7.9 HamerSoft.Victoria.Editor Namespace Reference

Classes

- class VictoriaWindow

  Victoria editor window which renders the importer.

7.10 HamerSoft.Victoria.EditorAudio Namespace Reference

Classes

- class EditorAudioSource

  Methods can be found in the UnityEditor.AudioUtil Class You can decompile it, or view the source here and implement more methods https://github.com/jamesjlinden/unity-decompiled/blob/master/UnityEditor/UnityEditor/AudioUtil.cs.

7.11 HamerSoft.Victoria.Loader Namespace Reference

7.12 HamerSoft.Victoria.Loader.Loader Namespace Reference

7.13 HamerSoft.Victoria.Tests Namespace Reference

7.14 HamerSoft.Victoria.Tests.Editor Namespace Reference

7.15 HamerSoft.Victoria.Tests.Editor.Helpers Namespace Reference

7.16 HamerSoft.Victoria.Tests.Runtime Namespace Reference

7.17 HamerSoft.Victoria.Ui Namespace Reference

Classes

- class Victoria

  Static factory for creating Victoria importer instances at runtime. Provides convenience methods for embedding the importer UI into an existing VisualElement hierarchy without needing to manage a Unity Editor window.

7.18 HamerSoft.Victoria.Ui.Elements Namespace Reference

Classes

- class VictoriaElement

  Root UI element of the Victoria importer. Renders a horizontal split-pane layout with a package asset tree on the left and an asset details / import destination panel on the right.

7.19 HamerSoft.Victoria.Ui.Elements.Nodes Namespace Reference

7.20 HamerSoft.Victoria.Ui.SleurEnPleur Namespace Reference

8 Class Documentation

8.1 HamerSoft.Victoria.Core.Extractor.Nodes.Asset Class Reference

Class representing an asset.
Inheritance diagram for HamerSoft.Victoria.Core.Extractor.Nodes.Asset:

8.2 HamerSoft.Victoria.EditorAudio.EditorAudioSource Class Reference

Methods can be found in the UnityEditor.AudioUtil Class You can decompile it, or view the source here and implement more methods https://github.com/jamesjlinden/unity-decompiled/blob/master/UnityEditor/UnityEditor/AudioUtil.cs.
Inheritance diagram for HamerSoft.Victoria.EditorAudio.EditorAudioSource:
Collaboration diagram for HamerSoft.Victoria.EditorAudio.EditorAudioSource:

Public Member Functions

- **EditorAudioSource** ()

  Initializes a new instance of EditorAudioSource and resolves all `UnityEditor.AudioUtil` methods via reflection.
- void Play (int startSample=0, bool loop=false)

  Plays the currently assigned clip from the given sample offset with an optional loop.
- void Play (AudioClip clip)

  Assigns clip as the active clip and immediately starts playback from the beginning.
- void SetClip (AudioClip clip)

  Sets the active clip. Stops any current playback before switching clips.
- void **Stop** ()

  Stops playback of the active clip. Does nothing if no clip is assigned.
- bool IsPlayingClip ()

  Returns whether the active clip is currently playing.
- void **PauseClip** ()

  Pauses playback of the active clip. Does nothing if no clip is assigned.
- void **ResumeClip** ()

  Resumes playback of a previously paused clip. Does nothing if no clip is assigned.
- void LoopClip (bool on)

  Enables or disables looping for the active clip.
- float GetClipPosition ()

  Gets the current playback position of the active clip in seconds.
- int GetClipSamplePosition ()

  Gets the current playback position of the active clip in samples.
- int GetSampleCount ()

  Gets the total number of samples in the active clip.
- int GetChannelCount ()

  Gets the number of audio channels in the active clip.
- int GetBitRate ()

  Gets the bit rate of the active clip in bits per second.
- int GetBitsPerSample ()

  Gets the bit depth (bits per sample) of the active clip.
- int GetFrequency ()

  Gets the sample rate (frequency) of the active clip in Hz.

- int GetSoundSize ()

  Gets the uncompressed size of the active clip's audio data in bytes.
- AudioCompressionFormat GetSoundCompressionFormat ()

  Gets the runtime audio compression format of the active clip.
- AudioCompressionFormat GetTargetPlatformSoundCompressionFormat ()

  Gets the audio compression format for the active clip on the current build target platform.
- float GetDuration ()

  Gets the duration of the active clip in seconds.
- void **Dispose** ()

  Stops playback and releases resources held by this instance.

Static Public Member Functions

- static void **StopAllClips** ()

  Stops all currently playing audio preview clips in the editor, regardless of which EditorAudioSource instance started them.

Properties

- bool **IsPlaying** `[get]`

  Flag whether the AudioSource is currently playing.

Properties inherited from HamerSoft.Victoria.Core.Audio.IAudioSource

8.2.1 Detailed Description

Methods can be found in the UnityEditor.AudioUtil Class You can decompile it, or view the source here and implement more methods https://github.com/jamesjlinden/unity-decompiled/blob/master/UnityEditor/UnityEditor/AudioUtil.cs.

8.2.2 Member Function Documentation

GetBitRate()

`int HamerSoft.Victoria.EditorAudio.EditorAudioSource.GetBitRate ( ) [inline]`
Gets the bit rate of the active clip in bits per second.

Returns

The bit rate in bps, or `0` if no clip is assigned.

GetBitsPerSample()

`int HamerSoft.Victoria.EditorAudio.EditorAudioSource.GetBitsPerSample ( ) [inline]`
Gets the bit depth (bits per sample) of the active clip.

Returns

The bits-per-sample depth, or `0` if no clip is assigned.

GetChannelCount()

`int HamerSoft.Victoria.EditorAudio.EditorAudioSource.GetChannelCount ( ) [inline]`
Gets the number of audio channels in the active clip.

Returns

The channel count (e.g. `1` for mono, `2` for stereo), or `0` if no clip is assigned.

GetClipPosition()

```
float HamerSoft.Victoria.EditorAudio.EditorAudioSource.GetClipPosition ( )  [inline]
```
Gets the current playback position of the active clip in seconds.

Returns

The playback position in seconds, or `0` if no clip is assigned.

GetClipSamplePosition()

```
int HamerSoft.Victoria.EditorAudio.EditorAudioSource.GetClipSamplePosition ( )  [inline]
```
Gets the current playback position of the active clip in samples.

Returns

The playback position as a sample index, or `0` if no clip is assigned.

GetDuration()

```
float HamerSoft.Victoria.EditorAudio.EditorAudioSource.GetDuration ( )  [inline]
```
Gets the duration of the active clip in seconds.

Returns

The duration in seconds as a float, or `0` if no clip is assigned.

GetFrequency()

```
int HamerSoft.Victoria.EditorAudio.EditorAudioSource.GetFrequency ( )  [inline]
```
Gets the sample rate (frequency) of the active clip in Hz.

Returns

The frequency in Hz, or `0` if no clip is assigned.

GetSampleCount()

```
int HamerSoft.Victoria.EditorAudio.EditorAudioSource.GetSampleCount ( )  [inline]
```
Gets the total number of samples in the active clip.

Returns

The sample count, or `0` if no clip is assigned.

GetSoundCompressionFormat()

```
AudioCompressionFormat HamerSoft.Victoria.EditorAudio.EditorAudioSource.GetSoundCompression↩
Format ( )  [inline]
```
Gets the runtime audio compression format of the active clip.

Returns

The AudioCompressionFormat of the clip, or AudioCompressionFormat.Vorbis if no clip is assigned.

GetSoundSize()

```
int HamerSoft.Victoria.EditorAudio.EditorAudioSource.GetSoundSize ( )  [inline]
```
Gets the uncompressed size of the active clip's audio data in bytes.

Returns

The sound size in bytes, or `0` if no clip is assigned.

GetTargetPlatformSoundCompressionFormat()

```
AudioCompressionFormat HamerSoft.Victoria.EditorAudio.EditorAudioSource.GetTargetPlatform↩
SoundCompressionFormat ( )  [inline]
```

Gets the audio compression format for the active clip on the current build target platform.

Returns

The target-platform AudioCompressionFormat, or AudioCompressionFormat.Vorbis if no clip is assigned.

IsPlayingClip()

```
bool HamerSoft.Victoria.EditorAudio.EditorAudioSource.IsPlayingClip ( )  [inline]
```

Returns whether the active clip is currently playing.

Returns

`true` if a clip is assigned and currently playing; `false` if no clip is assigned or playback has stopped.

LoopClip()

```
void HamerSoft.Victoria.EditorAudio.EditorAudioSource.LoopClip (
            bool on )  [inline]
```

Enables or disables looping for the active clip.

Parameters

on	`true` to enable looping; `false` to disable it.

Play() [1/2]

```
void HamerSoft.Victoria.EditorAudio.EditorAudioSource.Play (
            AudioClip clip )  [inline]
```

Assigns *clip* as the active clip and immediately starts playback from the beginning.

Parameters

clip	The AudioClip to assign and play.

Implements HamerSoft.Victoria.Core.Audio.IAudioSource.

Play() [2/2]

```
void HamerSoft.Victoria.EditorAudio.EditorAudioSource.Play (
            int startSample = 0,
            bool loop = false )  [inline]
```

Plays the currently assigned clip from the given sample offset with an optional loop.

Parameters

startSample	The sample index at which playback begins. Defaults to `0`.
loop	Whether the clip should loop. Defaults to `false`.

SetClip()

```
void HamerSoft.Victoria.EditorAudio.EditorAudioSource.SetClip (
```

```
            AudioClip clip )  [inline]
```

Sets the active clip. Stops any current playback before switching clips.

Parameters

clip	The AudioClip to set as the active clip.

The documentation for this class was generated from the following file:

- Editor/EditorAudioSource.cs

8.3 HamerSoft.Victoria.Core.Extractor.Extractor Class Reference

Extractor class to load a .unitypackage file into memory.

Static Public Member Functions

- static Folder Parse (FileInfo inputFile)

  Parse the .unitypackage file.

8.3.1 Detailed Description

Extractor class to load a .unitypackage file into memory.

8.3.2 Member Function Documentation

Parse()

```
static Folder HamerSoft.Victoria.Core.Extractor.Extractor.Parse (
            FileInfo inputFile )  [inline], [static]
```

Parse the .unitypackage file.

Parameters

inputFile	file reference

Returns

root folder of the unitypackage

The documentation for this class was generated from the following file:

- Runtime/Core/Extractor/Extractor.cs

8.4 HamerSoft.Victoria.Core.Extractor.Nodes.FileSystemNode Class Reference

A Node implementation that wraps an existing file-system entry. Constructed from a DirectoryInfo (non-leaf) or a FileInfo (leaf). Unlike package-extracted nodes, file-system nodes already exist on disk and cannot be written out.
Inheritance diagram for HamerSoft.Victoria.Core.Extractor.Nodes.FileSystemNode:
Collaboration diagram for HamerSoft.Victoria.Core.Extractor.Nodes.FileSystemNode:

Public Member Functions

- FileSystemNode (DirectoryInfo directoryInfo)

  Creates a non-leaf node representing a directory. Recursively builds child FileSystemNode instances for all files and subdirectories it contains.
- override Task WriteOut (string _)

  Not supported for file-system nodes — the entry already exists on disk.
- string FileExtension ()

  Gets the file extension of this node.

Public Member Functions inherited from HamerSoft.Victoria.Core.Extractor.Nodes.Node

- Task WriteOut (string rootPath)

  Write out a node (back) to disk.

Properties

- override bool **IsLeaf** `[get]`

  `true` *if this node represents a file;* `false` *if it represents a directory.*
- override string **DetailedName** `[get]`

  The display name of this node. For files this includes the file extension (e.g. `icon.png`*); for directories it is the bare folder name.*

Properties inherited from HamerSoft.Victoria.Core.Extractor.Nodes.Node

- string **Name** `[get, set]`

  Name of the node.
- string **Path** `[get, set]`

  Full path of the node.
- HashSet< Node > Children `[get, protected set]`

  Children of the node.
- bool **IsLeaf** `[get]`

  Whether the node is a leaf node.
- string **DetailedName** `[get]`

  Full file / folder name.
- Node **Parent** `[get, set]`

  The parent node.
- bool **HasChildren** `[get]`

  Check whether the node has children.

8.4.1 Detailed Description

A Node implementation that wraps an existing file-system entry. Constructed from a DirectoryInfo (non-leaf) or a FileInfo (leaf). Unlike package-extracted nodes, file-system nodes already exist on disk and cannot be written out.

8.4.2 Constructor & Destructor Documentation

FileSystemNode()

```
HamerSoft.Victoria.Core.Extractor.Nodes.FileSystemNode.FileSystemNode (
            DirectoryInfo directoryInfo )  [inline]
```

Creates a non-leaf node representing a directory. Recursively builds child FileSystemNode instances for all files and subdirectories it contains.

Parameters

directoryInfo	The directory to represent.

8.4.3 Member Function Documentation

FileExtension()

```
string HamerSoft.Victoria.Core.Extractor.Nodes.FileSystemNode.FileExtension ( )  [inline]
```

Gets the file extension of this node.

Returns

The extension including the leading dot (e.g. `.png`) for leaf nodes, or an empty string for directory nodes.

WriteOut()

```
override Task HamerSoft.Victoria.Core.Extractor.Nodes.FileSystemNode.WriteOut (
            string _ )  [inline]
```

Not supported for file-system nodes — the entry already exists on disk.

Parameters

↩ _↩	Unused root path parameter.

Returns

Never returns; always throws NotSupportedException.

Exceptions

NotSupportedException	Always thrown.

The documentation for this class was generated from the following file:

- Runtime/Core/Extractor/Nodes/FileSystemNode.cs

8.5 HamerSoft.Victoria.Core.Extractor.Nodes.Folder Class Reference

Class representing a folder of the imported .unityPackage.
Inheritance diagram for HamerSoft.Victoria.Core.Extractor.Nodes.Folder:
Collaboration diagram for HamerSoft.Victoria.Core.Extractor.Nodes.Folder:

Public Member Functions

- Folder (string name)

  Creates a new folder node with the given name. The node path is initialised to name and children are compared by name for deduplication.
- bool TryGetChildFolder (string folderName, out Folder child)

  Try to find a direct child folder by name.
- void AddChild (Node node)

  Add a new child Node to this folder.
- override Task **WriteOut** (string rootPath)

Public Member Functions inherited from HamerSoft.Victoria.Core.Extractor.Nodes.Node

- Task WriteOut (string rootPath)

  Write out a node (back) to disk.

Properties

- override bool **IsLeaf** `[get]`

  A folder is never a leaf, it can have children but does not have to.
- override string **DetailedName** `[get]`

  Folder name.

Properties inherited from HamerSoft.Victoria.Core.Extractor.Nodes.Node

- string **Name** `[get, set]`

  Name of the node.
- string **Path** `[get, set]`

  Full path of the node.
- HashSet< Node > Children `[get, protected set]`

  Children of the node.
- bool **IsLeaf** `[get]`

  Whether the node is a leaf node.
- string **DetailedName** `[get]`

  Full file / folder name.
- Node **Parent** `[get, set]`

  The parent node.
- bool **HasChildren** `[get]`

  Check whether the node has children.

8.5.1 Detailed Description

Class representing a folder of the imported .unityPackage.

8.5.2 Constructor & Destructor Documentation

Folder()

```
HamerSoft.Victoria.Core.Extractor.Nodes.Folder.Folder (
            string name ) [inline]
```

Creates a new folder node with the given name. The node path is initialised to *name* and children are compared by name for deduplication.

Parameters

name	The name of the folder.

8.5.3 Member Function Documentation

AddChild()

```
void HamerSoft.Victoria.Core.Extractor.Nodes.Folder.AddChild (
            Node node ) [inline]
```

Add a new child Node to this folder.

Parameters

node	reference to the node to add

TryGetChildFolder()

```
bool HamerSoft.Victoria.Core.Extractor.Nodes.Folder.TryGetChildFolder (
            string folderName,
            out Folder child ) [inline]
```

Try to find a direct child folder by name.

Parameters

folderName	Name of the child folder to find

Parameters

child	the child folder, or null

Returns

true if the direct child exists

The documentation for this class was generated from the following file:

- Runtime/Core/Extractor/Nodes/Folder.cs

8.6 HamerSoft.Victoria.Core.Audio.IAudioSource Interface Reference

AudioSource used for previewing audio data.
Inheritance diagram for HamerSoft.Victoria.Core.Audio.IAudioSource:

Public Member Functions

- void Play (AudioClip clip)

  Play the clip on the audio source.
- void Stop ()

  Stop the currently playing clip.

Properties

- bool **IsPlaying** `[get]`

  Flag whether the AudioSource is currently playing.

8.6.1 Detailed Description

AudioSource used for previewing audio data.

8.6.2 Member Function Documentation

Play()

```
void HamerSoft.Victoria.Core.Audio.IAudioSource.Play (
            AudioClip clip )
```

Play the clip on the audio source.
Stops the currently playing audio clip

Parameters

clip	clip to play

Implemented in HamerSoft.Victoria.EditorAudio.EditorAudioSource.

Stop()

```
void HamerSoft.Victoria.Core.Audio.IAudioSource.Stop ( )
```

Stop the currently playing clip.
If none is playing, this is a noop
Implemented in HamerSoft.Victoria.EditorAudio.EditorAudioSource.
The documentation for this interface was generated from the following file:

- Runtime/Core/Audio/IAudioSource.cs

8.7 HamerSoft.Victoria.Core.Extractor.Nodes.Node Class Reference

base class for representing nodes in the file-tree
Inheritance diagram for HamerSoft.Victoria.Core.Extractor.Nodes.Node:

Public Member Functions

- Task WriteOut (string rootPath)

  Write out a node (back) to disk.

Properties

- string **Name** `[get, set]`

  Name of the node.
- string **Path** `[get, set]`

  Full path of the node.
- HashSet< Node > Children `[get, protected set]`

  Children of the node.
- bool **IsLeaf** `[get]`

  Whether the node is a leaf node.
- string **DetailedName** `[get]`

  Full file / folder name.
- Node **Parent** `[get, set]`

  The parent node.
- bool **HasChildren** `[get]`

  Check whether the node has children.

8.7.1 Detailed Description

base class for representing nodes in the file-tree

8.7.2 Member Function Documentation

WriteOut()

```
Task HamerSoft.Victoria.Core.Extractor.Nodes.Node.WriteOut (
            string rootPath ) [abstract]
```

Write out a node (back) to disk.

Parameters

rootPath	Path where to write it to

Returns

awaitable IO task

8.7.3 Property Documentation

Children

```
HashSet<Node> HamerSoft.Victoria.Core.Extractor.Nodes.Node.Children [get], [protected set]
```

Children of the node.
always initialized
The documentation for this class was generated from the following file:

- Runtime/Core/Extractor/Nodes/Node.cs

8.8 HamerSoft.Victoria.UnityPackage Class Reference

Represents a parsed `.unitypackage` file. Exposes the package's asset tree and provides methods for loading individual assets on demand and importing them to disk. Dispose when done to release all cached Unity objects and the audio source.
Inherits IDisposable.

Public Member Functions

- async Task< T > LoadObject< T > (string id, byte[] data, Asset.Preview type, CancellationToken token)

  Asynchronously loads a Unity object of type T from raw asset data. The result is cached by id and type ; subsequent calls with the same key return the cached instance without re-loading.
- void **Dispose** ()

  Destroys all cached Unity objects — using `DestroyImmediate` in the Editor and `Destroy` at runtime — clears the cache, and disposes the audio source.

Static Public Member Functions

- static UnityPackage LoadFromPath (FileInfo selectedPackage, IAudioSource audioSource)

  Parses a `.unitypackage` file from disk and returns a ready-to-use UnityPackage.

Public Attributes

- readonly string **Name**

  The root folder name of the package, derived from the top-level Folder in the asset tree.

Properties

- Folder **Assets** `[get]`

  The root Folder of the package's asset tree, containing the full hierarchy of folders and assets parsed from the `.unitypackage` file.

8.8.1 Detailed Description

Represents a parsed `.unitypackage` file. Exposes the package's asset tree and provides methods for loading individual assets on demand and importing them to disk. Dispose when done to release all cached Unity objects and the audio source.

8.8.2 Member Function Documentation

LoadFromPath()

```
static UnityPackage HamerSoft.Victoria.UnityPackage.LoadFromPath (
            FileInfo selectedPackage,
            IAudioSource audioSource )  [inline], [static]
```

Parses a `.unitypackage` file from disk and returns a ready-to-use UnityPackage.

Parameters

selectedPackage	A FileInfo pointing to the `.unitypackage` file.
audioSource	The audio source used for previewing audio assets in the importer UI.

Returns

A fully parsed UnityPackage with its asset tree populated.

Exceptions

FileLoadException	Thrown if the package cannot be parsed.

LoadObject< T >()

```
async Task< T > HamerSoft.Victoria.UnityPackage.LoadObject< T > (
            string id,
            byte[] data,
            Asset::Preview type,
            CancellationToken token )  [inline]
```

Asynchronously loads a Unity object of type *T* from raw asset data. The result is cached by *id* and *type* ; subsequent calls with the same key return the cached instance without re-loading.

Template Parameters

T	The expected Unity object type (e.g. `Texture2D`, `AudioClip`).

Parameters

id	A unique identifier for the asset within this package.
data	The raw byte data of the asset to load.
type	The preview type hint used to determine how to decode *data* .
token	Cancellation token to abort the load operation.

Returns

The loaded object of type *T* , or `null` if loading failed.

The documentation for this class was generated from the following file:

- Runtime/UnityPackage.cs

8.9 HamerSoft.Victoria.Ui.Victoria Class Reference

Static factory for creating Victoria importer instances at runtime. Provides convenience methods for embedding the importer UI into an existing VisualElement hierarchy without needing to manage a Unity Editor window.

Classes

- class VictoriaRuntimeImporter

  A self-contained VisualElement that renders the full Victoria import UI at runtime. Owns the lifecycle of the loaded UnityPackage and the underlying VictoriaElement; dispose to release all cached Unity objects.

Static Public Member Functions

- static VictoriaRuntimeImporter Create (VisualElement parent, string source)

  Creates a VictoriaRuntimeImporter using the default import destination (`Application.persistentData`↩ `Path/Victoria/Imports`).
- static VictoriaRuntimeImporter Create (VisualElement parent, string source, string destination)

  Creates a VictoriaRuntimeImporter targeting the specified destination directory, creating it on disk if it does not already exist.

8.9.1 Detailed Description

Static factory for creating Victoria importer instances at runtime. Provides convenience methods for embedding the importer UI into an existing VisualElement hierarchy without needing to manage a Unity Editor window.

8.9.2 Member Function Documentation

Create() [1/2]

```
static VictoriaRuntimeImporter HamerSoft.Victoria.Ui.Victoria.Create (
            VisualElement parent,
            string source )  [inline], [static]
```

Creates a VictoriaRuntimeImporter using the default import destination (`Application.persistentData↩Path/Victoria/Imports`).

Parameters

parent	The VisualElement to attach the importer UI to.
source	Absolute path to the `.unitypackage` file to import.

Returns

A fully initialised VictoriaRuntimeImporter added to *parent* .

Create() [2/2]

```
static VictoriaRuntimeImporter HamerSoft.Victoria.Ui.Victoria.Create (
            VisualElement parent,
            string source,
            string destination )  [inline], [static]
```

Creates a VictoriaRuntimeImporter targeting the specified destination directory, creating it on disk if it does not already exist.

Parameters

parent	The VisualElement to attach the importer UI to.
source	Absolute path to the `.unitypackage` file to import.
destination	Absolute path to the directory where assets will be written.

Returns

A fully initialised VictoriaRuntimeImporter added to *parent* .

The documentation for this class was generated from the following file:

- Runtime/Ui/Victoria.cs

8.10 HamerSoft.Victoria.Ui.Elements.VictoriaElement Class Reference

Root UI element of the Victoria importer. Renders a horizontal split-pane layout with a package asset tree on the left and an asset details / import destination panel on the right.
Inherits VisualElement, and HamerSoft.Victoria.Ui.SleurEnPleur.IDragParent.

Public Member Functions

- VictoriaElement (UnityPackage unityPackage, DirectoryInfo destination, Action onFinishedImport)

  Builds the split-pane importer UI, initialises the import manifest and node factory, and subscribes to node focus events.

8.10.1 Detailed Description

Root UI element of the Victoria importer. Renders a horizontal split-pane layout with a package asset tree on the left and an asset details / import destination panel on the right.

8.10.2 Constructor & Destructor Documentation

VictoriaElement()

```
HamerSoft.Victoria.Ui.Elements.VictoriaElement.VictoriaElement (
            UnityPackage unityPackage,
            DirectoryInfo destination,
            Action onFinishedImport )  [inline]
```

Builds the split-pane importer UI, initialises the import manifest and node factory, and subscribes to node focus events.

Parameters

unityPackage	The parsed package whose asset tree is displayed.
destination	The root directory to which assets will be imported.
onFinishedImport	Callback invoked when the import operation completes.

The documentation for this class was generated from the following file:

- Runtime/Ui/Elements/VictoriaElement.cs

8.11 HamerSoft.Victoria.Ui.Victoria.VictoriaRuntimeImporter Class Reference

A self-contained VisualElement that renders the full Victoria import UI at runtime. Owns the lifecycle of the loaded UnityPackage and the underlying VictoriaElement; dispose to release all cached Unity objects.
Inherits VisualElement, and IDisposable.

Public Member Functions

- void **Dispose** ()

  Disposes the loaded UnityPackage — destroying all cached Unity objects — and tears down the VictoriaElement UI.

8.11.1 Detailed Description

A self-contained VisualElement that renders the full Victoria import UI at runtime. Owns the lifecycle of the loaded UnityPackage and the underlying VictoriaElement; dispose to release all cached Unity objects.
The documentation for this class was generated from the following file:

- Runtime/Ui/Victoria.cs

8.12 HamerSoft.Victoria.Editor.VictoriaWindow Class Reference

Victoria editor window which renders the importer.
Inherits EditorWindow.

Static Public Member Functions

- static void **SpawnWindow** ()

  Entry point for showing the import window in the editor.

8.12.1 Detailed Description

Victoria editor window which renders the importer.
The documentation for this class was generated from the following file:

- Editor/VictoriaWindow.cs

Index